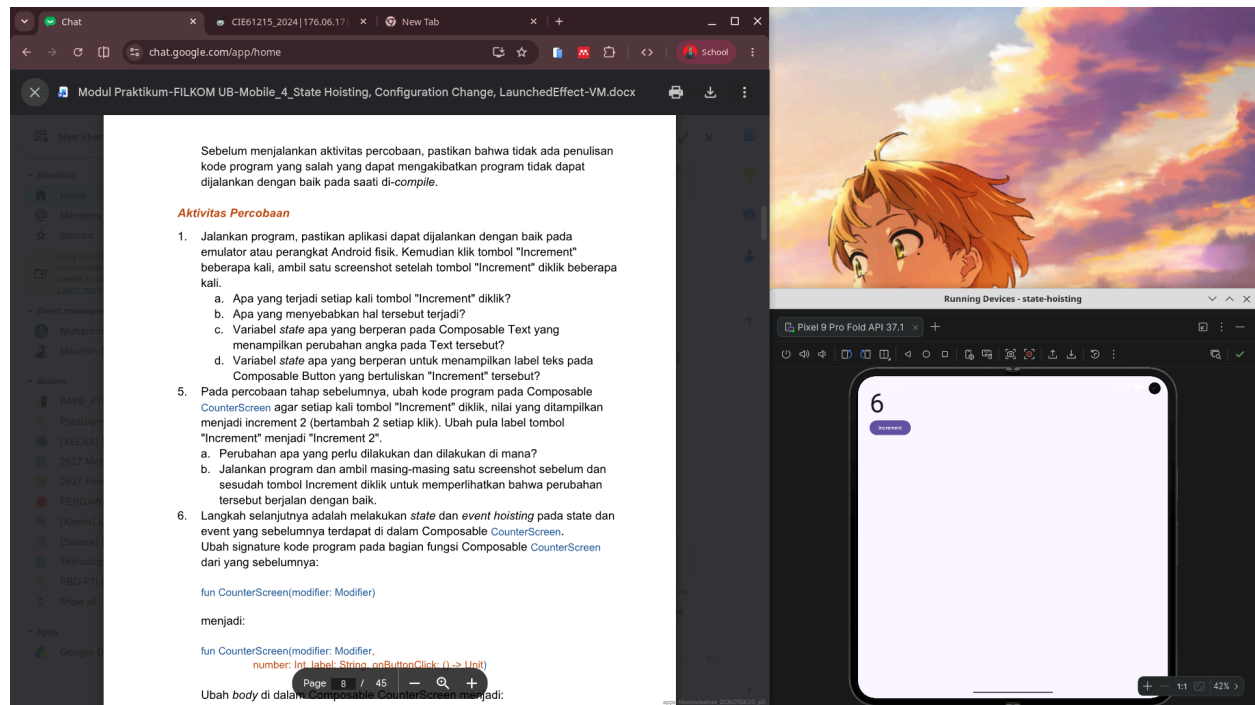


1.5 Percobaan

1.5 State Hoisting



1. Apa yang terjadi setiap kali tombol "Increment" diklik?

Angka yang ditampilkan pada Text bertambah 1 setiap kali tombol diklik (0, 1, 2, 3, dst).

1b. Apa yang menyebabkan hal tersebut terjadi?

Karena onClick pada Button menjalankan `number++`, yang mengubah nilai state number (`mutableStateOf`). Perubahan state ini memicu recomposition pada `Text(text = "$number", ...)` sehingga tampilan diperbarui.

1c. Variabel state yang berperan pada Composable Text (angka)

number — dideklarasikan dengan `var number by remember { mutableStateOf(0) }`.

1d. Variabel state yang berperan pada label tombol "Increment"

label — dideklarasikan dengan `val label by remember { mutableStateOf("Increment") }`.

5a. Perubahan apa yang perlu dilakukan dan di mana?

- Ubah `onClick = { number++ }` menjadi `onClick = { number += 2 }`.
- Ubah nilai awal label dari "Increment" menjadi "Increment 2".

7a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, secara fungsional dan visual perilakunya identik — hanya cara pengelolaan state-nya yang berbeda (sekarang di-hoist ke onCreate).

7b. Di bagian kode program manakah saat ini proses increment dilakukan?

Di dalam onCreate(), pada lambda onClick = { number++ } saat memanggil CounterScreen. Composable CounterScreen sendiri hanya memanggil onClick tanpa tahu logika di dalamnya (event di-hoist ke parent).

7c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button “Increment” dideklarasikan?

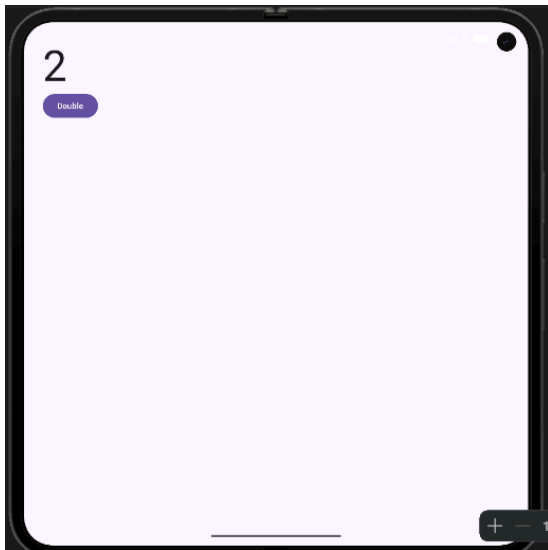
State number dideklarasikan di onCreate() (var number by remember { mutableStateOf(0) }), lalu diteruskan sebagai parameter number: Int dan label: String ke CounterScreen. Composable CounterScreen tidak lagi punya state sendiri (stateless).

8d. Kode setelah 8a–8c:

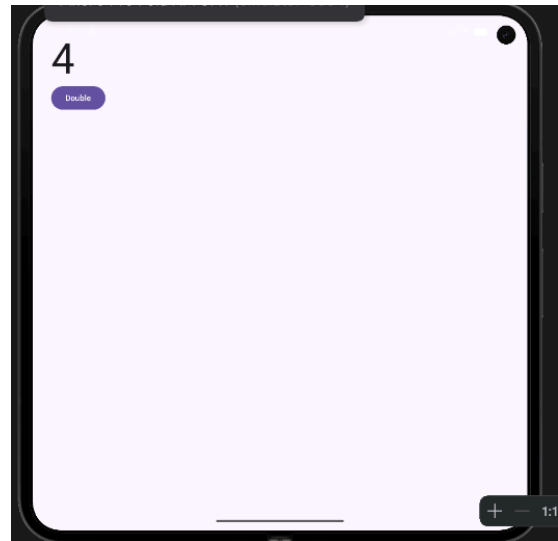
```
var number by remember { mutableStateOf(1) }
CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = { number *= 2 }
)
```

9a.

Before



After



9b. Di bagian manakah perubahan tahap 7 dan 8 dilakukan?

Seluruhnya dilakukan di `onCreate()` — pada deklarasi state awal (`mutableStateOf`), label yang dikirim, dan logika lambda `onButtonClick`.

9c. Apakah ada perubahan di dalam Composable `CounterScreen` itu sendiri?

Tidak ada. Kode `CounterScreen` (signature dan body-nya) tidak diubah sama sekali dari tahap 6 hingga tahap 8 — ia hanya menerima parameter baru dan menjalankan callback yang diberikan.

10. Makna jawaban nomor 9 terkait manfaat state hoisting

Ini membuktikan manfaat utama state hoisting: ketika terjadi perubahan kebutuhan fungsional (logika bisnis — dari increment 1 jadi double, label berubah, nilai awal berubah), composable UI (`CounterScreen`) tidak perlu disentuh sama sekali. Semua perubahan logika cukup dilakukan di level parent (`onCreate`). Ini menunjukkan separation of concern yang baik: `CounterScreen` murni komponen presentasi (stateless, reusable) yang independen dari logika bisnis yang bisa berubah-ubah.

4.5.2 Perubahan Konfigurasi

2b. Berapa angka yang ditampilkan setelah rotasi (pakai remember)? Mengapa?

Angka akan kembali ke nilai awal (1), bukan angka terakhir sebelum rotasi. Ini terjadi karena perubahan konfigurasi (rotasi layar) menyebabkan Android menghancurkan dan membuat ulang Activity. remember hanya menyimpan state selama composable berada dalam composition yang sama; saat Activity dibuat ulang, composition lama hilang sepenuhnya sehingga state ikut ter-reset ke nilai inisialisasi.

3b. Berapa angka yang ditampilkan setelah rotasi (pakai rememberSaveable)? Mengapa?

Angka tetap sama seperti sebelum rotasi (nilai terakhir yang sudah di-double beberapa kali). Ini karena rememberSaveable menyimpan nilai state ke dalam Bundle (saved instance state) sebelum Activity dihancurkan, dan mengembalikannya saat Activity dibuat ulang.

3c. Fungsi rememberSaveable yang membedakannya dari remember

remember hanya mempertahankan state selama recomposition dalam composition yang sama, dan akan hilang jika Activity dihancurkan/dibuat ulang (misal rotasi layar). rememberSaveable mempertahankan state baik saat recomposition maupun saat Activity dibuat ulang akibat perubahan konfigurasi, dengan cara menyimpan nilainya ke Bundle (saved instance state), selama tipe datanya bisa dimasukkan ke Bundle (atau menggunakan custom Saver untuk tipe kompleks).

4. Contoh kasus nyata penggunaan rememberSaveable

Contoh: form input multi-field (misalnya form registrasi dengan banyak TextField seperti nama, email, nomor telepon) atau state checkbox/filter pada halaman pencarian. Jika pengguna sudah mengisi sebagian form lalu secara tidak sengaja memutar perangkat (portrait ke landscape), data yang sudah diisi tidak akan hilang. Manfaatnya: pengalaman pengguna (UX) jadi lebih baik karena pengguna tidak perlu mengulang input dari awal setiap kali terjadi perubahan konfigurasi seperti rotasi layar atau perubahan bahasa sistem.

4.5.3 LaunchedEffect

Aktivitas Percobaan 1

1a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double"; apa yang ditampilkan di dalam layar?

Snackbar langsung muncul dengan pesan "Number 1 is shown!".

1b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena `LaunchedEffect(number)` dijalankan otomatis sekali saat composable pertama kali masuk ke composition, terlepas dari ada tidaknya interaksi pengguna. Karena number sudah punya nilai awal (1), blok `showSnackBar` langsung dieksekusi begitu composable tampil di layar.

1c. Apa yang memicu pesan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicu adalah masuknya composable ke dalam composition (initial composition), ditangkap oleh baris:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackBar("Number $number is shown!")  
}
```

2a. Klik tombol "Double" sekali — apa yang terjadi?

Angka pada layar berubah menjadi 2 (digandakan), dan SnackBar baru muncul lagi dengan pesan "Number 2 is shown!".

2b. Mengapa hal itu terjadi?

Karena number adalah key dari `LaunchedEffect(number)`. Setiap kali nilai number berubah, Compose akan membatalkan coroutine `LaunchedEffect` yang lama dan menjalankan ulang blok di dalamnya dengan nilai number yang baru, sehingga SnackBar ditampilkan kembali dengan pesan terbaru.

Aktivitas Percobaan 2

1c. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double", apa yang ditampilkan di dalam layar?

Angka awal (1) ditampilkan terlebih dahulu beserta SnackBar "Number 1 is shown!" — karena `LaunchedEffect(number)` tetap berjalan seperti biasa saat composition pertama kali terjadi.

1d. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Angka berubah menjadi angka acak hasil `Repo.getData()` (antara 100–999), dan SnackBar baru muncul menampilkan angka acak tersebut.

1e. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Dipicu oleh `LaunchedEffect(Unit) { number = Repo.getData() }`. Blok ini berjalan sekali saat composable pertama kali masuk komposisi, memanggil fungsi `suspend getData()` yang men-delay 2 detik lalu mengembalikan angka acak. Begitu number di-assign nilai baru, state berubah → memicu recomposition sekaligus memicu ulang `LaunchedEffect(number)` yang menampilkan Snackbar dengan angka baru tersebut.

3a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)`: blok akan dijalankan ulang setiap kali nilai state yang dijadikan key berubah (coroutine lama dibatalkan, lalu dijalankan ulang dengan nilai terbaru).

`LaunchedEffect(Unit)`: karena Unit adalah nilai yang tidak pernah berubah, blok ini hanya dijalankan sekali sepanjang umur composable berada dalam composition (umumnya dipakai untuk operasi one-time seperti fetch data awal).

3b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

`LaunchedEffect(state)` dijalankan pertama kali saat composable masuk komposisi, dan setiap kali nilai state berubah selama composable masih berada dalam composition.

`LaunchedEffect(Unit)` hanya dijalankan satu kali, yaitu saat composable pertama kali masuk komposisi, dan tidak akan dijalankan ulang kecuali composable tersebut keluar dan masuk kembali ke komposisi (misalnya karena navigasi atau recomposition struktural yang menghapus composable-nya).